Houdini Solaris / USD Pipeline Guide

1. Purpose of This Document

This document is a **team styleguide** for working with USD in Houdini Solaris.

It is written for artists who may be **new to USD** but already understand 3D workflows from Unreal, Maya, Cinema 4D, Blender, or similar tools.

The goal is not to explain every technical detail of USD. The goal is to give the team:

- A clear mental model of how USD works
- A consistent way of structuring projects
- Practical rules for daily production
- Naming conventions that are readable by humans and scripts
- Enough understanding to avoid common pipeline mistakes

This guide defines:

- How files are structured
- How assets and shots are organized
- How artists collaborate
- What each department owns
- How data flows through the pipeline
- How HIP files and USD files relate to each other

1.1 What This Document Is NOT

This guide is not:

- A deep USD technical manual
- A node-by-node Houdini tutorial
- A temporary onboarding plan
- A replacement for production judgement

👉 It is a **long-term production rulebook** for how the team works.

1.2 How to Use This Guide

- New to USD → read sections 1–4 carefully
- Starting a task → follow the relevant workflow sections
- Naming files → use section 10
- Setting up folders → use section 16
- Something is broken → use the debugging checklist

---

2. Core Principles

These concepts are the foundation of the entire pipeline. If you understand this section, the rest of the guide will make sense.

2.1 USD vs Traditional Scene Files

Traditional Workflow

In many traditional workflows, such as Unreal, Maya, or Cinema 4D, artists often work inside a single large scene file:

- A level
- A scene
- A project file
- A map

That file contains many types of work at once: layout, cameras, animation, lighting, materials, and sometimes rendering setup.

This can work well, but it creates problems:

- Multiple artists need access to the same file
- Files must often be locked
- Merge conflicts are difficult or impossible
- It is easy to overwrite other people's work
- It is hard to know what changed and why

2.2 USD Workflow

USD works differently.

Instead of one scene file, a scene is built from **many smaller files**. Each file contributes part of the final result.

Example:

```
neon_0010.usda
 ├── neon_0010_layout.usda
 ├── neon_0010_anim.usda
 ├── neon_0010_fx.usda
 └── neon_0010_lighting.usda
```

Each department publishes its own layer. The final shot is composed from those layers.

👉 You are not editing one shared scene. You are contributing your own layer to a composed scene.

2.3 Key Mental Shift

Traditional thinking:

> "I am editing the scene."

USD thinking:

> “I am adding my part of the scene.”

This is the most important mindset shift when moving to Solaris/USD.

2.4 What Is a USD Layer?

A USD file does not need to contain a complete scene.

It can contain only:

- Geometry
- Materials
- Material assignments
- Asset placement
- Animation
- FX caches
- Lighting
- Render settings

These partial files are called **layers**.

Example:

```
neon_0010_layout.usda    → where assets are placed
neon_0010_anim.usda      → how objects move
neon_0010_lighting.usda  → how the shot is lit
```

When composed together, they create the final shot.

👉 Each layer adds information without destroying previous work.

2.5 HIP vs USD

In this pipeline, there are two important file types:

HIP Files

HIP files are Houdini work files.

They are:

- Artist workspaces
- Node graphs
- Temporary and changeable
- Used to generate USD

HIP files are not the final production data.

USD Files

USD files are the published production data.

They are:

- Shared across the team
- Referenced by other departments
- Used to build shots
- The source of truth

The workflow is:

```
HIP → Publish → USD
```

👉 USD is always the **source of truth**.

---

3. Core Rules

These rules are non-negotiable.

1. One department owns one USD layer.
2. One task should live in one HIP file.
3. Never edit another department's USD layer.
4. Always publish USD before handing off work.

5. Never rely on HIP files for sharing production data.
6. Published USD paths should remain stable.
7. Use SVN for version history and coordination.
8. Use naming conventions exactly.

👉 If a workflow breaks one of these rules, it should be discussed before it becomes habit.

---

4. Pipeline Overview

4.1 High-Level Flow

```
Assets → Layout → Animation → FX → Lighting → Render
```

Each step publishes USD that feeds into the next step.

4.2 Asset Flow

```
Model → Lookdev → Rig → Assembly → Published Asset
```

Rigging introduces hierarchy, deformation, and animation controls. It builds on top of model geometry and prepares the asset for animation.

Example:

```
char_robot_model.usdc
char_robot_lookdev.usda
char_robot.usda
```

The final assembled asset, such as `char_robot.usda`, is what shots should reference.

4.3 Shot Flow

```
Layout → Animation → FX → Lighting → Final Shot
```

Example:

```
neon_0010.usda
 ├── neon_0010_layout.usda
 ├── neon_0010_anim.usda
 ├── neon_0010_fx.usda
 └── neon_0010_lighting.usda
```

👉 Final shot files are composed from department layers.

---

5. Sequence and Shot Logic

Projects may contain multiple sequences, and each sequence may contain multiple shots.

The production hierarchy is:

```
Project → Sequence → Shot → Department
```

5.1 Sequence Codes

Sequences use short project-defined codes.

Examples:

```
neon
trmc
elec
```

Rules:

- Use 3–5 lowercase letters
- Keep codes short and readable

- Use the same code everywhere
- Do not mix uppercase and lowercase in filenames

> Note: Production may display sequence codes as uppercase in schedules or documents (`NEON`, `TRMC`, `ELEC`), but filenames and folders should use lowercase (`neon`, `trmc`, `elec`) for consistency and machine readability.

5.2 Shot Codes

Shots use four-digit numbers.

Examples:

```
0010
0020
0030
```

Recommended numbering increments by 10 so new shots can be inserted later.

5.3 Shot Context

The full shot context is:

```
<sequence>_<shot>
```

Examples:

```
neon_0010
neon_0020
trmc_0010
```

This context is used in filenames:

```
neon_0010_anim.usda
neon_0010_lighting_maria_v001.hip
```

5.4 Folder Structure vs Filename Structure

Folders split sequence and shot for clarity:

```
/shots/neon/0010/
/shots/neon/0020/
/shots/trmc/0010/
```

Filenames combine them for uniqueness:

```
neon_0010_layout.usda
neon_0010_anim.usda
neon_0010.usda
```

👉 Folders organize the project. Filenames identify the shot uniquely.

---

6. Ownership and Responsibility

Every USD layer must have:

- One responsible department
- One responsible artist at a time

6.1 Department Ownership

Department	Owns	Does Not Own
Modeling	Geometry	Materials, shots, lighting
Lookdev	Materials and assignments	Shot lighting
Rigging	Hierarchy, deformation, controls	Geometry topology, materials
Assembly	Final asset package	Shot layout
Layout	Asset placement, cameras	Asset geometry
Animation	Motion	Layout, asset lookdev
FX	Simulations, FX caches	Animation timing unless agreed

Department	Owns	Does Not Own
Lighting	Lights, render settings	Permanent asset materials

6.2 Ownership Transfer

Ownership transfer must be explicit.

Examples:

- An animator takes over another animator’s shot.
- A senior artist fixes someone else’s HIP file.
- Lookdev hands an asset to assembly.

When this happens:

1. SVN update.
2. Open the latest file.
3. Save a new HIP version.
4. Publish USD.
5. Commit with a clear message.

7. Dependency Rules

USD is a dependency chain.

Example:

```
char_robot.usda
        ↓
neon_0010_layout.usda
        ↓
neon_0010_anim.usda
        ↓
neon_0010_lighting.usda
        ↓
neon_0010.usda
```

7.1 If Upstream Data Changes

If an upstream file changes, downstream artists must verify their work.

Example: if layout changes, animation and lighting must check their files.

You MUST:

- SVN update
- Reload references in Solaris
- Check that your layer still works
- Republish if needed

You MUST NOT:

- Copy upstream data locally
- Ignore broken references
- Fix problems by duplicating assets into your HIP

---

8. HIP Files

8.1 What HIP Files Are

HIP files are artist workspaces used to create USD.

They contain:

- Houdini node graphs
- SOP work
- LOP/Solaris networks
- Work-in-progress setups
- USD ROPs for publishing

HIP files are not final pipeline data.

8.2 What HIP Files Should Import

A HIP file should import or reference published USD only.

Examples:

```
Layout HIP imports asset USD
Animation HIP imports layout USD
Lighting HIP imports animation USD
```

Do not reference another artist's HIP file.

8.3 What HIP Files Should Export

Each HIP should export one main USD layer.

Examples:

```
neon_0010_layout_ina_v001.hip      → neon_0010_layout.usda
neon_0010_anim_erik_v001.hip       → neon_0010_anim.usda
neon_0010_lighting_maria_v001.hip  → neon_0010_lighting.usda
```

👉 One HIP = one responsibility = one main USD output.

8.4 Where HIP Files Live

HIP files live inside the department folder:

```
/shots/neon/0010/layout/hip/
/shots/neon/0010/anim/hip/
/shots/neon/0010/lighting/hip/
```

Asset HIP files live inside asset department folders:

```
/assets/char_robot/model/hip/
/assets/char_robot/lookdev/hip/
/assets/char_robot/assembly/hip/
```

8.5 HIP Versioning

HIP files are versioned:

```
neon_0010_anim_erik_v001.hip
neon_0010_anim_erik_v002.hip
```

Published USD filenames usually remain stable:

```
neon_0010_anim.usda
```

Why:

- Other departments can reference a stable path
- SVN tracks history
- The final shot does not need to be updated for every artist save

---

8.6 Are HIP Files Shared?

Default rule:

👉 HIP files are artist-owned, not collaboration files.

They can be opened by another artist only for:

- Handoff
- Debugging
- Support
- Training

If someone else takes over, they save a new version.

8.7 Multiple Artists in One HIP

Default rule:

👉 One artist edits a HIP file at a time.

Do not:

- Have two artists edit the same HIP simultaneously
- Use one shared master HIP for a shot
- Use HIP files as the collaboration layer

Correct collaboration happens through USD layers.

8.8 Modeling Hierarchy Rule (Geometry Only)

Modeling defines **geometry only**, not production hierarchy. Published model USD files must contain a **clean, flat structure** under a single root prim and a `/geo` scope so downstream departments can build consistent, intentional hierarchy later.

Rules

- Do not define production hierarchy in modeling
- Do not rely on parenting inside the modeling DCC
- Object names define logical parts of the asset
- Exported USD should be flattened during publish
- Geometry must live under `/geo`
- Naming must follow pipeline conventions

Parenting in Blender or other DCCs is considered **temporary and non-authoritative**. The exporter will ignore or flatten hierarchy unless explicitly defined by pipeline rules.

⚠️ Hierarchy is a pipeline contract. Once introduced, it affects animation, layout, and lookdev — so it must only be created in the correct stage.

Example (Published Modeling USD Prim Layout)

```
/char_robot
    /geo
        /body
        /arm_l
        /arm_r
        /head
```

Responsibility

- Modeling is responsible for:
  - Geometry
  - Clean topology
  - Naming
  - Material slot assignment
- Modeling is NOT responsible for:
  - Rig structure
  - Animation hierarchy
  - Control systems

Downstream Hierarchy

Any meaningful hierarchy must be created in a downstream layer (e.g. rigging).

```
Modeling (flat) → Rigging (hierarchy) → Animation
```

---

9. Materials Strategy

Materials can live in two places depending on their purpose.

9.1 Asset Materials

Location:

```
/assets/<asset>/lookdev/materials/
```

Use when:

- The material is specific to one asset
- It depends on that asset's UVs or textures
- It represents the final look of that asset

Examples:

```
/assets/char_robot/lookdev/materials/char_robot_paint.usda
/assets/char_robot/lookdev/materials/char_robot_metal.usda
```

9.2 Asset Lookdev Assignment Layer

Location:

```
/assets/<asset>/lookdev/usd/
```

Example:

```
/assets/char_robot/lookdev/usd/char_robot_lookdev.usda
```

This file should primarily contain:

- Material bindings
- References to material definitions
- Lookdev opinions for the asset

It should not be treated as the only place where material definitions live.

9.3 Library Materials

Location:

```
/library/materials/
```

Use when:

- The material is generic and reusable
- It is not tied to one asset
- It can be used as a building block

Examples:

```
/library/materials/metal.usda
/library/materials/plastic.usda
/library/materials/glass.usda
```

9.4 Core Material Rule

```
Library = reusable material building blocks
Asset lookdev = final asset look
```

Example:

```
/library/materials/metal.usda
        ↓
/assets/char_robot/lookdev/materials/char_robot_metal.usda
        ↓
/assets/char_robot/lookdev/usd/char_robot_lookdev.usda
```

---

10. Naming Conventions

Naming must be **consistent, predictable, and machine-readable**.

This means:

- No ambiguity
- No personal naming styles
- Files can be parsed by scripts
- Optional fields are handled consistently

10.1 General Naming Rules

- Use lowercase for filenames and folders
- Use `_` between major filename tokens
- Do not use spaces
- Do not use special characters
- Do not use words like `final`, `latest`, `new`, or `test` as version substitutes
- Keep names short but descriptive

Allowed characters:

```
a-z 0-9 _ - .
```

Use hyphens only inside optional descriptors if needed.

10.2 Sequence and Shot Naming

Sequence and shot context:

```
<sequence>_<shot>
```

Where:

- `<sequence>` = 3–5 lowercase letters
- `<shot>` = 4 digits

Examples:

```
neon_0010
trmc_0020
elec_0030
```

10.3 Pipeline Step Tokens

Use these exact task tokens:

Step	Token
Modeling	`model`
Lookdev	`lookdev`
Assembly	`assembly`
Layout	`layout`
Animation	`anim`
FX	`fx`
Lighting	`lighting`

Do not invent alternate tokens like `lgt`, `ani`, `geo`, or `mat` unless the styleguide is updated.

10.4 Published Asset USD Naming

Pattern:

```
<asset>_<task>.<ext>
```

Examples:

```
char_robot_model.usdc
char_robot_lookdev.usda
char_robot_assembly.usda
```

Final assembled asset may use the clean asset name:

```
char_robot.usda
prop_crate.usda
```

Rules:

- Published asset USD filenames are stable
- Do not include artist names
- Do not include version numbers in the main published filename

---

10.5 Published Shot USD Naming

Department layer pattern:

```
<sequence>_<shot>_<task>.<ext>
```

Examples:

```
neon_0010_layout.usda
neon_0010_anim.usda
neon_0010_fx.usda
neon_0010_lighting.usda
```

Final shot composition pattern:

```
<sequence>_<shot>.usda
```

Example:

```
neon_0010.usda
```

10.6 HIP Naming

Shot HIP pattern:

```
<sequence>_<shot>_<task>[_<descriptor>]_<artist>_v###.hip
```

Examples:

```
neon_0010_anim_erik_v001.hip
neon_0010_anim_blocking_erik_v002.hip
neon_0010_layout_ina_v001.hip
```

Asset HIP pattern:

```
<asset>_<task>[_<descriptor>]_<artist>_v###.hip
```

Examples:

```
char_robot_model_alex_v001.hip
char_robot_lookdev_maria_v001.hip
char_robot_assembly_alex_v001.hip
```

10.7 Optional Descriptor Rules

The descriptor is optional.

With descriptor:

```
neon_0010_anim_blocking_erik_v001.hip
```

Without descriptor:

```
neon_0010_anim_erik_v001.hip
```

If the descriptor is omitted, remove the token and its underscore.

Correct:

```
neon_0010_anim_erik_v001.hip
```

Incorrect:

```
neon_0010_anim__erik_v001.hip
```

👉 There should never be empty tokens or double underscores.

10.8 Token Boundaries and Parsing

A parser should not blindly split a filename on every underscore.

Instead, parsing is anchored by known tokens:

- Known task tokens: `model`, `lookdev`, `assembly`, `layout`, `anim`, `fx`, `lighting`
- Known version pattern: `v###`
- Known shot pattern: `<sequence>_<shot>`

Example:

```
char_robot_model.usdc
```

Parsed as:

```
context = char_robot
task    = model
ext     = usdc
```

Example:

```
neon_0010_anim_blocking_erik_v002.hip
```

Parsed as:

```
sequence   = neon
shot       = 0010
task       = anim
descriptor = blocking
artist     = erik
version    = v002
ext        = hip
```

👉 Context names such as `char_robot` may contain underscores because parsing is anchored from known task/version patterns.

10.9 Versioning Rules

- Always use `v###`
- Start at `v001`
- Increment on meaningful saves or handoffs
- Do not use `final`, `latest`, or dates as versions

Incorrect:

```
v1
v01
latest
final
```

Correct:

```
v001
v002
v003
```

10.10 Filename Validation Patterns

These regex patterns can be used for simple validation tools or pre-commit checks.

Published Asset USD

```
^[a-z0-9]+(?:_[a-z0-9]+)*_(model|lookdev|assembly)\.(usd|usda|usdc)$
```

Also allowed for final assembled assets:

```
^[a-z0-9]+(?:_[a-z0-9]+)*\.(usd|usda|usdc)$
```

Published Shot USD Layer

```
^[a-z]{3,5}_[0-9]{4}_(layout|anim|fx|lighting)\.(usd|usda|usdc)$
```

Final Shot Composition USD

```
^[a-z]{3,5}_[0-9]{4}\.(usd|usda|usdc)$
```

HIP Working Files

```
^[a-z0-9]+(?:_[a-z0-9]+)*_(model|lookdev|assembly|layout|anim|fx|lighting)(?:_[a-z0-9]+
(?:-[a-z0-9]+)*)?_[a-z]+_v[0-9]{3}\.hip$
```

This HIP regex covers both asset and shot HIPs because both use a context followed by a known task token.

10.11 Good vs Bad Naming

Bad:

```
final_render_v2.hip
new_anim_latest_FIX.usda
robotStuff_v3_FINAL.usd
```

```
neon_0010_anim_test_reallyFinal_v7.hip
myFile.hip
```

Good:

```
neon_0010_anim_erik_v001.hip
neon_0010_anim_blocking_erik_v002.hip
char_robot_model.usdc
char_robot_lookdev.usda
neon_0010_lighting.usda
```

Before vs after:

Bad	Good
`final_anim_v2.hip`	`neon_0010_anim_erik_v002.hip`
`robotStuff.usd`	`char_robot_model.usdc`
`lighting_new_v5.hip`	`neon_0010_lighting_maria_v005.hip`
`test_render_latest.usd`	`neon_0010_lighting.usda`

👉 If a filename cannot be understood without opening it, it is wrong.

10.12 Naming Cheat Sheet

Published asset USD:

```
<asset>_<task>.usda
<asset>.usda
```

Published shot USD:

```
<sequence>_<shot>_<task>.usda
<sequence>_<shot>.usda
```

Shot HIP:

```
<sequence>_<shot>_<task>[_<descriptor>]_<artist>_v###.hip
```

Asset HIP:

```
<asset>_<task>[_<descriptor>]_<artist>_v###.hip
```

Examples:

```
char_robot_model.usdc
char_robot.usda
neon_0010_layout.usda
neon_0010_anim_erik_v001.hip
```

---

11. Paths and Environment Variables

Never hardcode absolute local paths.

Use project-relative paths or environment variables.

Recommended variables:

```
$PROJECT
$ASSETS
$SHOTS
$LIBRARY
```

Examples:

```
$ASSETS/char_robot/assembly/usd/char_robot.usda
$SHOTS/neon/0010/layout/usd/neon_0010_layout.usda
$LIBRARY/materials/metal.usda
```

Do not use local machine paths like:

```
C:/Users/artist/Desktop/project/...
```

12. Publishing

A USD file is considered **published** only when all conditions are met:

- It exports without errors
- It loads in a fresh HIP
- It uses valid paths
- It follows naming conventions
- It is committed to SVN
- Downstream departments can reference it

If any of these fail, it is not published.

12.1 Published USD Paths

Published USD filenames should usually remain stable.

Example:

```
neon_0010_anim.usda
```

Avoid publishing main layers as:

```
neon_0010_anim_v001.usda
neon_0010_anim_v002.usda
```

SVN tracks file history. Stable USD paths make references easier and reduce broken dependencies.

Optional version archives can exist in a `versions/` folder if needed, but the main published file should remain stable.

13. Source Control (SVN)

SVN is used for:

- Version history
- Backup
- Coordination
- Rollback

13.1 Commit These

Commit:

- HIP files
- Published USD files
- Textures
- Small project tools
- Documentation

13.2 Usually Do Not Commit These

Do not commit:

- Render outputs
- Local Houdini backup files
- Crash files
- Temporary caches
- Personal scratch files

Simulation caches should be decided per project. Heavy caches may need external storage.

13.3 Daily SVN Workflow

1. SVN Update
2. Open HIP
3. Work
4. Publish USD
5. Test the USD
6. SVN Commit

Commit messages should be short and clear.

Examples:

```
Layout: updated camera framing for neon_0010
Anim: adjusted robot timing
Lookdev: updated robot paint material
```

---

14. Debugging Checklist

When something breaks, check in this order:

1. Does the file exist?
2. Is the path correct?
3. Is the Reference LOP pointing to the right USD?
4. Is the layer included in the final composition?
5. Is the layer order correct?
6. Did an upstream prim path change?
7. Did someone publish without committing?
8. Are you accidentally looking at an old cached version?

👉 Most USD problems are path, reference, or layer-order problems.

---

15. Performance Guidelines

Keep the pipeline simple and lightweight.

Rules:

- Prefer referencing USD over rebuilding data
- Keep LOP networks clean
- Avoid unnecessary layers
- Avoid heavy SOP cooking inside shot lighting files
- Cache simulations intentionally
- Do not duplicate assets into shots
- Avoid one giant master HIP

---

16. Quick Start for Daily Use

16.1 Daily Workflow

Every artist should do this:

1. SVN Update
2. Open your HIP
3. Work in your department only
4. Publish USD
5. Check the published USD
6. SVN Commit

16.2 Minimal Shot Files

A simple shot may contain:

```
neon_0010_layout.usda
neon_0010_anim.usda
```

```
neon_0010_lighting.usda
neon_0010.usda
```

With FX:

```
neon_0010_fx.usda
```

16.3 Roles

Assets

Assets define what things are.

They contain:

- Model
- Lookdev
- Assembly

Assets are built once and reused in shots.

⚠️ Assets should not be permanently modified inside shot files.

Rigging

Rigging defines how an asset moves and deforms.

It introduces:

- Hierarchy
- Deformation systems
- Animation controls

Rigging builds on top of model geometry and does not modify the underlying topology.

Output

Rigging publishes a USD layer:

```
<asset>_rig.usda
```

This layer adds structure and animation interfaces to the asset.

Relationship to Modeling

```
Model → flat geometry
Rig → hierarchy + behavior
```

Hierarchy is introduced in rigging, not modeling.

This ensures that animation structure is consistent, intentional, and owned by the correct department.

⚠️ Rigging must not change topology. Any topology changes belong in modeling and must be republished before rigging continues.

Layout

Layout defines where things are.

Layout owns:

- Asset placement
- Cameras
- Shot composition

Animation

Animation defines how things move.

Animation owns:

- Character motion
- Object motion
- Timing on top of layout

FX

FX defines simulation and effects work.

FX owns:

- Simulated elements
- FX caches
- FX USD layers

Lighting

Lighting defines how the shot is rendered.

Lighting owns:

- Lights
- Render settings
- Shot-level visual adjustments

16.4 Simple Mental Model

```
Assets    → what things are
Layout    → where they are
Animation → how they move
FX        → what effects happen
Lighting  → how they look
```

---

17. Full Project Example (Folder Structure)

This section is formatted as an expandable folder tree with consistent metadata callouts (Owner / Warning / Used by).

▼ 📁 **project_robot_short/** *Root project folder. Everything for the project lives here.*

> ⚠️ Do not place random files at root

▼ 📁 **assets/** *Reusable assets (characters, props, environments).*

> ⚠️ Never put shot-specific work here

▼ 📁 **char_robot/** *Main character asset*

▼ 📁 **model/** *Geometry creation*

> 👤 Owner: Modeling

▼ 📁 **hip/** *Work files*

- `char_robot_model_alex_v001.hip`

▼ 📁 **usd/** *Published geometry*

> 🔄 Used by: Lookdev, Assembly

- `char_robot_model.usdc`

▼ 📁 **lookdev/** *Materials, textures, shading*

> 👤 Owner: Lookdev

▼ 📁 **hip/**

⚠️ Local work only

- `char_robot_lookdev_maria_v001.hip`

▼ 📁 **materials/** *Material definitions (shaders, networks)*

🔄 Can be reused within this asset

- `char_robot_paint.usda`
- `char_robot_metal.usda`

▼ 📁 **usd/** *Material assignment layer*

⚠️ Do NOT define materials here

🔄 Used by: Assembly

- `char_robot_lookdev.usda`

▼ 📁 **textures/** *Source textures*

⚠️ Do not duplicate in shots

- `char_robot_basecolor.exr`
- `char_robot_roughness.exr`

▼ 📁 **rig/** *Rigging defines hierarchy, deformation, and control systems for animation.*

👤 Owner: Rigging

- 📁 **hip/** *Work files*
  - `char_robot_rig_<artist>_v###.hip`
- 📁 **usd/** *Published rig layer*

🔄 Used by: Animation

  - `char_robot_rig.usda`

- 📁 **assembly/** *Final asset package*

👤 Owner: Assembly

  - 📁 **hip/**
    - `char_robot_assembly_alex_v001.hip`
  - 📁 **usd/**

🔄 Used by: Layout

    - `char_robot.usda`

- 📁 **shots/** *All shot work lives here*

⚠️ Never store reusable assets here

  - 📁 **neon/** *Sequence folder*
    - 📁 **0010/** *Shot folder*

▼ 📁 **layout/** *Placement and cameras*

> 👤 Owner: Layout

▼ 📁 **hip/**

- `neon_0010_layout_ina_v001.hip`

▼ 📁 **usd/**

> 🔄 Used by: Animation

- `neon_0010_layout.usda`

▼ 📁 **anim/** *Animation work*

> 👤 Owner: Animation

▼ 📁 **hip/**

- `neon_0010_anim_erik_v001.hip`

▼ 📁 **usd/**

> 🔄 Used by: FX, Lighting

- `neon_0010_anim.usda`

▼ 📁 **fx/** *Simulations*

> 👤 Owner: FX

▼ 📁 **hip/**

- `neon_0010_fx_nora_v001.hip`

▼ 📁 **usd/**

> 🔄 Used by: Lighting

- `neon_0010_fx.usda`

▼ 📁 **cache/** *Simulation caches*

- `sim.0001.vdb`
- `sim.0002.vdb`

▼ 📁 **lighting/** *Lighting and rendering*

> 👤 Owner: Lighting

▼ 📁 **hip/**

- `neon_0010_lighting_maria_v001.hip`

▼ 📁 **usd/**

> 🔄 Used by: Render

- `neon_0010_lighting.usda`

▼ 📁 **usd/** *Final shot composition*

> ⚠️ Do not edit manually

- `neon_0010.usda`

▼ 📁 **library/** *Reusable shared assets*

▼ 📁 **materials/**

- `metal.usda`
- `plastic.usda`

▼ 📁 **lights/**

- `studio_rig.usda`

▼ 📁 **houdini/** *Tools and configs*

▼ 📁 **otls/**

- `studio_tools.hda`

▼ 📁 **docs/** *Documentation*

- `pipeline_guide.md`

---

18. Real Example: Asset Creation + Parallel Shot Work:

This example shows how a small team creates an asset, publishes it to USD, and then uses that asset in a shot where multiple artists work in parallel.

Project: `project_robot_short`

Sequence: `neon`

Shot: `0010`

Shot context: `neon_0010`

Artists:

- Alex — Modeling / Asset Assembly
- Maria — Lookdev / Lighting
- Ina — Layout
- Erik — Animation

18.1 Asset Creation

Step 1 — Modeling

Alex creates:

```
/assets/char_robot/model/hip/char_robot_model_alex_v001.hip
```

Publishes:

```
/assets/char_robot/model/usd/char_robot_model.usdc
```

This contains geometry and basic prim hierarchy. It does not contain final lookdev or shot animation.

Step 2 — Lookdev

Maria creates:

```
/assets/char_robot/lookdev/hip/char_robot_lookdev_maria_v001.hip
```

Material definitions:

```
/assets/char_robot/lookdev/materials/char_robot_paint.usda
/assets/char_robot/lookdev/materials/char_robot_metal.usda
```

Material assignment layer:

```
/assets/char_robot/lookdev/usd/char_robot_lookdev.usda
```

Step 3 — Rigging

Rigging creates:

```
/assets/char_robot/rig/hip/char_robot_rig_alex_v001.hip
```

Publishes:

```
/assets/char_robot/rig/usd/char_robot_rig.usda
```

This adds hierarchy, deformation, and animation controls without changing the underlying model topology.

Step 4 — Assembly

Alex creates:

```
/assets/char_robot/assembly/hip/char_robot_assembly_alex_v001.hip
```

Publishes final asset:

```
/assets/char_robot/assembly/usd/char_robot.usda
```

👉 Layout references `char_robot.usda`, not model or lookdev files directly.

18.2 Shot Production

Step 4 — Layout

Ina creates:

```
/shots/neon/0010/layout/hip/neon_0010_layout_ina_v001.hip
```

References:

```
/assets/char_robot/assembly/usd/char_robot.usda
/assets/prop_crate/assembly/usd/prop_crate.usda
```

Publishes:

```
/shots/neon/0010/layout/usd/neon_0010_layout.usda
```

Step 5 — Animation

Erik creates:

```
/shots/neon/0010/anim/hip/neon_0010_anim_erik_v001.hip
```

References:

```
/shots/neon/0010/layout/usd/neon_0010_layout.usda
```

Publishes:

```
/shots/neon/0010/anim/usd/neon_0010_anim.usda
```

Step 6 — Lighting

Maria creates:

```
/shots/neon/0010/lighting/hip/neon_0010_lighting_maria_v001.hip
```

References:

```
/shots/neon/0010/anim/usd/neon_0010_anim.usda
```

Publishes:

```
/shots/neon/0010/lighting/usd/neon_0010_lighting.usda
```

Step 7 — Final Shot Composition

Final composed shot:

```
/shots/neon/0010/usd/neon_0010.usda
```

Layer stack:

```
neon_0010.usda
 ├── neon_0010_layout.usda
 ├── neon_0010_anim.usda
 └── neon_0010_lighting.usda
```

18.3 What Happens When Something Changes?

If the robot model updates:

1. Modeling republishes `char_robot_model.usdc`
2. Lookdev checks material assignments
3. Assembly republishes `char_robot.usda` if needed
4. Layout updates references
5. Animation and Lighting verify their work

If layout updates:

1. Layout republishes `neon_0010_layout.usda`
2. Animation and Lighting SVN update
3. They reload references
4. They verify their layers still work

👉 Nobody copies assets into the shot. Everyone references published USD.

18.4 Visual Pipeline Diagram

Asset Creation Flow

```
char_robot_model_alex_v001.hip
        ↓
```

```
char_robot_model.usdc
        ↓
char_robot_lookdev_maria_v001.hip
        ↓
materials/*.usda + char_robot_lookdev.usda
        ↓
char_robot_rig_alex_v001.hip
        ↓
char_robot_rig.usda
        ↓
char_robot_assembly_alex_v001.hip
        ↓
char_robot.usda (FINAL ASSET)
```

Shot Production Flow

```
char_robot.usda
        ↓
neon_0010_layout_ina_v001.hip
        ↓
neon_0010_layout.usda
        ↓
neon_0010_anim_erik_v001.hip
        ↓
neon_0010_anim.usda
        ↓
neon_0010_lighting_maria_v001.hip
        ↓
neon_0010_lighting.usda
        ↓
neon_0010.usda (FINAL SHOT)
```

Parallel Work Concept

```
neon_0010_layout.usda
       ↓
neon_0010_anim.usda
       ↓
neon_0010_lighting.usda
       ↓
neon_0010.usda
```

Each department:

- Reads previous USD
- Adds its own layer
- Does not modify upstream data

👉 Think of USD like layers in Photoshop: each artist adds a layer, but no one paints directly on someone else's layer.

18.5 Bad Workflows

Do not:

- Reference `char_robot_model.usdc` directly in layout when `char_robot.usda` exists
- Edit asset geometry inside shot animation
- Define permanent asset materials inside shot lighting
- Work multiple artists inside one HIP file at the same time
- Pass HIP files around instead of publishing USD

---

19. Final Principles

Keep the pipeline:

- Simple
- Predictable
- Consistent

If unsure:

- Rebuild from published USD
- Do not hack around problems
- Ask who owns the layer before editing
- Keep stable published paths

End of guide